TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

Modern Software Specification

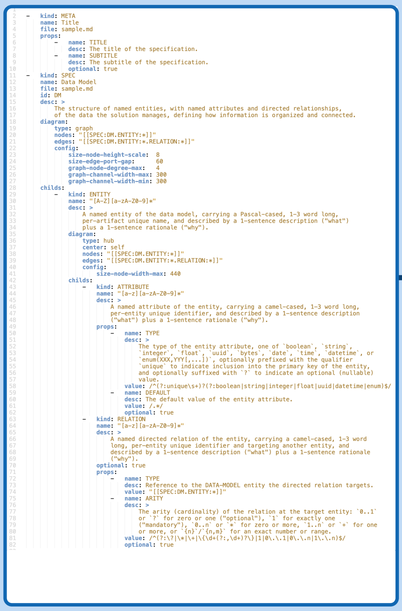
Spec Book

Hierarchical Object Model
Versatile Object to Markdown Mapping
Wiki-Style Object Linking

Strict Validation with Custom Schema
Object Model Diagram Visualisation
CLI, MCP, and API Interface

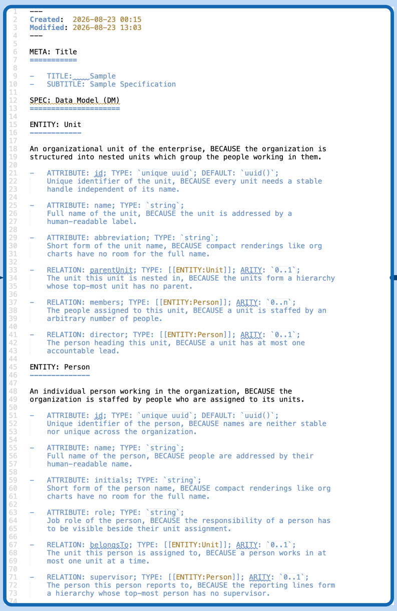
AST Exports for AI/LLMs
HTML Export for Developers
PDF Export for Customers

SpecBook
SCHEMA Model
(YAML)



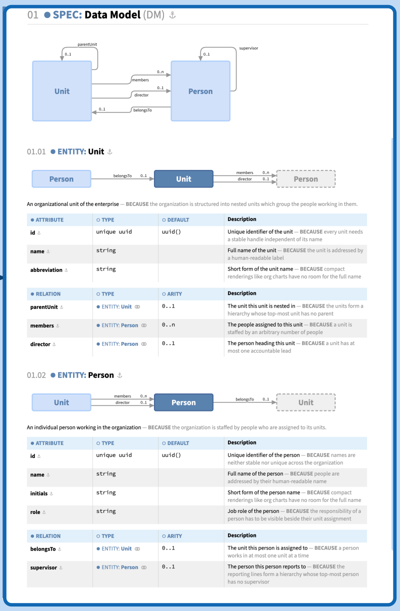
```yaml

-   kind: META
    name: Title
    file: sample.md
    props:
        -   name: TITLE
            desc: The title of the specification.
        -   name: SUBTITLE
            desc: The subtitle of the specification.
            optional: true
-   kind: SPEC
    name: Data Model
    file: sample.md
    id: DM
    desc: >
        The structure of named entities, with named attributes and directed relationships,
        of the data the solution manages, defining how information is organized and connected.
    diagram:
        type: graph
        nodes: "[[SPEC:DM.ENTITY:*]]"
        edges: "[[SPEC:DM.ENTITY:*.RELATION:*]]"
        config:
            size-node-height-scale:  8
            size-edge-port-gap:      60
            graph-node-degree-max:   4
            graph-channel-width-max: 300
            graph-channel-width-min: 300
    childs:
        -   kind: ENTITY
            name: "[A-Z][a-zA-Z0-9]*"
            desc: >
                A named entity of the data model, carrying a Pascal-cased, 1-3 word long,
                per-artifact unique name, and described by a 1-sentence description ("what")
                plus a 1-sentence rationale ("why").
            diagram:
                type: hub
                center: self
                nodes: "[[SPEC:DM.ENTITY:*]]"
                edges: "[[SPEC:DM.ENTITY:*.RELATION:*]]"
                config:
                    size-node-width-max: 440
            childs:
                -   kind: ATTRIBUTE
                    name: "[a-z][a-zA-Z0-9]*"
                    desc: >
                        A named attribute of the entity, carrying a camel-cased, 1-3 word long,
                        per-entity unique identifier, and described by a 1-sentence description
                        ("what") plus a 1-sentence rationale ("why").
                    props:
                        -   name: TYPE
                            desc: >
                                The type of the entity attribute, one of `boolean`, `string`,
                                `integer`, `float`, `uuid`, `bytes`, `date`, `time`, `datetime`, or
                                `enum(XXX,YYY[,...])`, optionally prefixed with the qualifier
                                `unique` to indicate inclusion into the primary key of the entity,
                                and optionally suffixed with `?` to indicate an optional (nullable)
                                value.
                            value: /^(?:unique\s+)?(?:boolean|string|integer|float|uuid|datetime|enum)$/
                        -   name: DEFAULT
                            desc: The default value of the entity attribute.
                            value: /.*/
                            optional: true
                -   kind: RELATION
                    name: "[a-z][a-zA-Z0-9]*"
                    desc: >
                        A named directed relation of the entity, carrying a camel-cased, 1-3 word
                        long, per-entity unique identifier and targeting another entity, and
                        described by a 1-sentence description ("what") plus a 1-sentence rationale
                        ("why").
                    optional: true
                    props:
                        -   name: TYPE
                            desc: Reference to the DATA-MODEL entity the directed relation targets.
                            value: "[[SPEC:DM.ENTITY:*]]"
                        -   name: ARITY
                            desc: >
                                The arity (cardinality) of the relation at the target entity: `0..1`
                                or `?` for zero or one ("optional"), `1` for exactly one
                                ("mandatory"), `0..n` or `*` for zero or more, `1..n` or `+` for one
                                or more, or `{n}`/`{n,m}` for an exact number or range.
                            value: /^(?:\?|\*|\+|\{\d+(?:,\d+)?\}|1|0\.\.1|0\.\.n|1\.\.n)$/
                            optional: true
```

SpecBook
SPEC Model
(Markdown)



```markdown
---
Created:  2026-08-23 00:15
Modified: 2026-08-23 13:03
---

META: Title
===========

-   TITLE:    Sample
-   SUBTITLE: Sample Specification

SPEC: Data Model (DM)
=====================

ENTITY: Unit
------------

An organizational unit of the enterprise, BECAUSE the organization is
structured into nested units which group the people working in them.

-   ATTRIBUTE: id; TYPE: `unique uuid`; DEFAULT: `uuid()`;
    Unique identifier of the unit, BECAUSE every unit needs a stable
    handle independent of its name.

-   ATTRIBUTE: name; TYPE: `string`;
    Full name of the unit, BECAUSE the unit is addressed by a
    human-readable label.

-   ATTRIBUTE: abbreviation; TYPE: `string`;
    Short form of the unit name, BECAUSE compact renderings like org
    charts have no room for the full name.

-   RELATION: parentUnit; TYPE: [[ENTITY:Unit]]; ARITY: `0..1`;
    The unit this unit is nested in, BECAUSE the units form a hierarchy
    whose top-most unit has no parent.

-   RELATION: members; TYPE: [[ENTITY:Person]]; ARITY: `0..n`;
    The people assigned to this unit, BECAUSE a unit is staffed by an
    arbitrary number of people.

-   RELATION: director; TYPE: [[ENTITY:Person]]; ARITY: `0..1`;
    The person heading this unit, BECAUSE a unit has at most one
    accountable lead.

ENTITY: Person
--------------

An individual person working in the organization, BECAUSE the
organization is staffed by people who are assigned to its units.

-   ATTRIBUTE: id; TYPE: `unique uuid`; DEFAULT: `uuid()`;
    Unique identifier of the person, BECAUSE names are neither stable
    nor unique across the organization.

-   ATTRIBUTE: name; TYPE: `string`;
    Full name of the person, BECAUSE people are addressed by their
    human-readable name.

-   ATTRIBUTE: initials; TYPE: `string`;
    Short form of the person name, BECAUSE compact renderings like org
    charts have no room for the full name.

-   ATTRIBUTE: role; TYPE: `string`;
    Job role of the person, BECAUSE the responsibility of a person has
    to be visible beside their unit assignment.

-   RELATION: belongsTo; TYPE: [[ENTITY:Unit]]; ARITY: `0..1`;
    The unit this person is assigned to, BECAUSE a person works in at
    most one unit at a time.

-   RELATION: supervisor; TYPE: [[ENTITY:Person]]; ARITY: `0..1`;
    The person this person reports to, BECAUSE the reporting lines form
    a hierarchy whose top-most person has no supervisor.
```

SpecBook
SPEC Export
(AST, HTML, PDF)



01 ● **SPEC: Data Model** (DM)

01.01 ● **ENTITY: Unit**

An organizational unit of the enterprise — **BECAUSE** the organization is structured into nested units which group the people working in them.

● ATTRIBUTE	○ TYPE	○ DEFAULT	Description
id	unique uuid	uuid()	Unique identifier of the unit — **BECAUSE** every unit needs a stable handle independent of its name
name	string		Full name of the unit — **BECAUSE** the unit is addressed by a human-readable label
abbreviation	string		Short form of the unit name — **BECAUSE** compact renderings like org charts have no room for the full name

● RELATION	○ TYPE	○ ARITY	Description
parentUnit	● ENTITY: **Unit**	0..1	The unit this unit is nested in — **BECAUSE** the units form a hierarchy whose top-most unit has no parent
members	● ENTITY: **Person**	0..n	The people assigned to this unit — **BECAUSE** a unit is staffed by an arbitrary number of people
director	● ENTITY: **Person**	0..1	The person heading this unit — **BECAUSE** a unit has at most one accountable lead

01.02 ● **ENTITY: Person**

An individual person working in the organization — **BECAUSE** the organization is staffed by people who are assigned to its units.

● ATTRIBUTE	○ TYPE	○ DEFAULT	Description
id	unique uuid	uuid()	Unique identifier of the person — **BECAUSE** names are neither stable nor unique across the organization
name	string		Full name of the person — **BECAUSE** people are addressed by their human-readable name
initials	string		Short form of the person name — **BECAUSE** compact renderings like org charts have no room for the full name
role	string		Job role of the person — **BECAUSE** the responsibility of a person has to be visible beside their unit assignment

● RELATION	○ TYPE	○ ARITY	Description
belongsTo	● ENTITY: **Unit**	0..1	The unit this person is assigned to — **BECAUSE** a person works in at most one unit at a time
supervisor	● ENTITY: **Person**	0..1	The person this person reports to — **BECAUSE** the reporting lines form a hierarchy whose top-most person has no supervisor

In modern Software Engineering, a software specification is increasingly treated as a structured model instead of a free-form document. A specification becomes a hierarchical graph of typed objects, such as entities, attributes, and relations, written in a plain-text format like **Markdown** and linked through explicit cross-references. This Specification-as-Code approach puts specifications under version control, makes changes reviewable as “diffs”, and keeps every fact in exactly one place.

A central idea is separating rules from content. A schema, the meta-model, defines which kinds of objects a specification may contain, how they nest, and what values their properties may take. The specification itself must conform to that schema. This allows automated validation of structure, references, and consistency, so a specification is checked rather than merely written, much like source code against its type system.

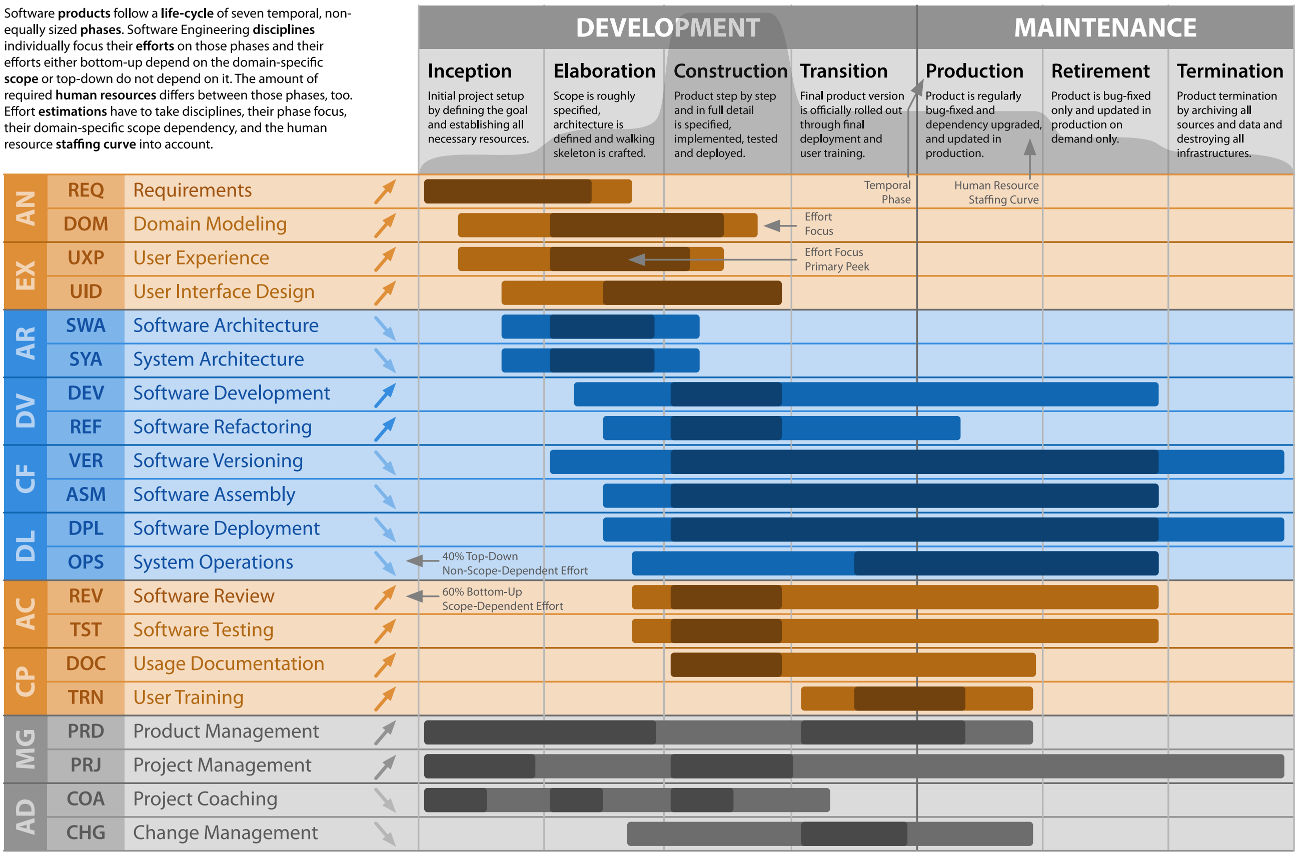
The specification becomes the **Single Source of Truth**: one specification model feeds many views for different audiences. These include navigable documentation for developers, binding documents for customers, and machine-readable syntax trees for AI agents. Diagrams can even be derived from the model instead of being drawn by hand, so they always match the text. SpecBook is an example of this modern approach.

Questions

- What does **Single Source of Truth** mean in the context of a **specification**?

Software Engineering Efforts

Software **products** follow a **life-cycle** of seven temporal, non-equally sized **phases**. Software Engineering **disciplines** individually focus their **efforts** on those phases and their efforts either bottom-up depend on the domain-specific **scope** or top-down do not depend on it. The amount of required **human resources** differs between those phases, too. Effort **estimations** have to take disciplines, their phase focus, their domain-specific scope dependency, and the human resource **staffing curve** into account.


DEVELOPMENT
MAINTENANCE
Inception
Initial project setup by defining the goal and establishing all necessary resources.
Elaboration
Scope is roughly specified, architecture is defined and walking skeleton is crafted.
Construction
Product step by step and in full detail is specified, implemented, tested and deployed.
Transition
Final product version is officially rolled out through final deployment and user training.
Production
Product is regularly bug-fixed and dependency upgraded, and updated in production.
Retirement
Product is bug-fixed only and updated in production on demand only.
Termination
Product termination by archiving all sources and data and destroying all infrastructures.
Temporal Phase
Human Resource Staffing Curve
Effort Focus
Effort Focus Primary Peek
40% Top-Down Non-Scope-Dependent Effort
60% Bottom-Up Scope-Dependent Effort
AN
REQ Requirements
DOM Domain Modeling
EX
UXP User Experience
UID User Interface Design
AR
SWA Software Architecture
SYA System Architecture
DV
DEV Software Development
REF Software Refactoring
CF
VER Software Versioning
ASM Software Assembly
DL
DPL Software Deployment
OPS System Operations
AC
REV Software Review
TST Software Testing
CP
DOC Usage Documentation
TRN User Training
MG
PRD Product Management
PRJ Project Management
AD
COA Project Coaching
CHG Change Management

Software products follow a **life-cycle** of seven temporal, non-equally sized **phases**. Software Engineering **disciplines** individually focus their **efforts** on those phases, and their efforts either bottom-up depend on the domain-specific scope, or their efforts top-down do not depend on it. The amount of required **human resources** differs between those phases, too.

Effort estimations have to take into account disciplines, their phase focus, their domain-specific scope dependency, and the human resource staffing curve.

Furthermore, the seven sequential phases, especially, do not conflict with agile process models: agile time **periods** (named “sprints” in Scrum) merely subdivide the individual phases.

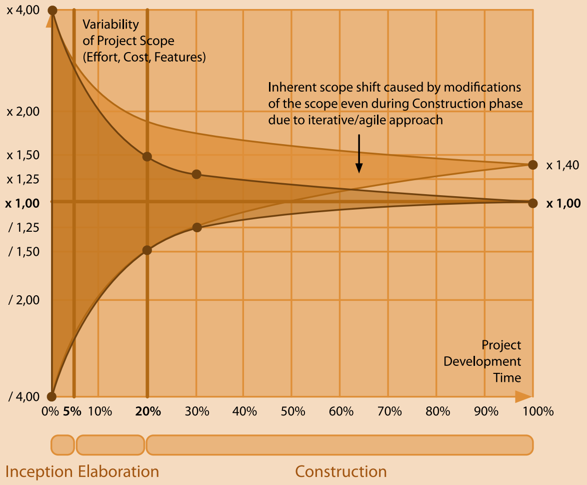
Questions

? What is the Software Engineering **phase** called, which has the greatest personnel requirements and in which primarily the functionalities are realized?

Uncertainty & Elaboration

Intellectual Content: Version 1.0.2 (2022-10-10), Authored 2021-2020 by Dr. Ralf S. Engelschall based on concepts from Steve McDonnell
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Cone of Uncertainty



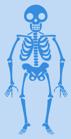
The ***Cone of Uncertainty*** (*Steve McConnell*, 2006) tells how the variability of the project scope (measured in Effort, Cost or Features) in Software Development changes over time. Initially, it usually is within the range of +/- 400% of the final scope.

The early development phases Inception and Elaboration especially have to ensure that within the first 20% of the project, the variability is reduced noticeably to just +/- 50%. During the initial iterations of the Construction phase within the first 30% of the project, the variability usually can be further reduced to about +/- 25%.

For iterative/agile approaches, experience showed that during the Construction phase inherently the final scope further shifts by about + 40% due to the just step-by-step learned required details of the required solution. This especially has to be taken into account for estimations.

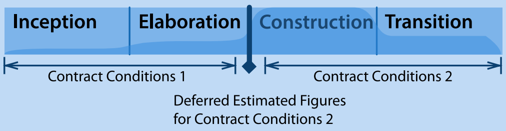
Essential Elaboration Phase

Walking Skeleton:



The ***Walking Skeleton*** (or *Technical Breakthrough*) is the design and implementation of the bare technical foundation of an application, still *without* any domain-specific functionalities. It is made during the Elaboration phase with the primary purpose to establish a stable integration of all technical aspects (libraries, frameworks, build procedures, etc) onto which the domain-specific functionalities later can be successively put onto.

Agile Fixed-Price Contracts:



The ***Agile Fixed-Price*** is an agile variant of a fixed-price contract, *not* a fixed-price project with an agile development process.

There are two important inherent aspects:

First, the contract contains two types of conditions: one (usually *Time & Material* but fixed duration based) for the Inception and Elaboration phases in order to make experiences and to gather necessary figures, and one (usually Fixed-User-Story and/or Fixed-Price based) for the Construction and Transition phases based on deferred estimated figures, gathered in the Elaboration phase.

Second, the Fixed-Price aspect of the contract is actually based on an amount of User-Stories (resulting in costs by multiplying them with either an average hourly rate of an engineer or individual rates based on engineer job levels), which the customer can 1:1 *exchange* during the project for different deliverables.

The crux of an Agile Fixed-Price contract is: first, during the Inception and Elaboration phases the supplier can shrink the *Cone of Uncertainty* and this way its risks dramatically, and second, during the Construction and Transition phases the customer still remains flexible in scope.

The **Cone of Uncertainty** tells how the variability of the project scope (measured in Effort, Cost or Features) in Software Development changes over time. The early development phases Inception and Elaboration especially have to ensure that the variability is reduced noticeably.

For iterative/agile approaches, experience showed that during the Construction phase, inherently, the final scope further shifts due to the just step-by-step learned, required details of the required solution.

The Elaboration phase is especially important for the creation of the **Walking Skeleton**, where all the technical integrations of libraries, frameworks, build procedures, etc., are done without already implementing any domain-specific functionalities.

Because of the **Cone of Uncertainty**, **Agile Fixed-Price** project contracts usually differentiate between the early phases Inception and Elaboration and the main phases Construction and Transition. The contract conditions of the latter usually depend on figures that can only be seriously estimated at the end of the Elaboration phase.

Questions

- What is especially developed in the project phase “Elaboration”?

Effort Estimations

Estimation & Variability

Three-Point Estimation and Estimation Variability Classes:

$e = (b + 4 \times m + w) / 6$ expected effort (weighted average)
$s = (w - b) / 6$ standard deviation (effort variation)

b: best-case (optimistic)
m: most-likely (realistic)
w: worst-case (pessimistic)

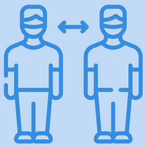
Insane Variability:	+/- 10%
Very Good Variability:	+/- 15%
Good Variability:	+/- 20%
Acceptable Variability:	+/- 25%

Conversion & Normalization

1. Ask Estimater:
"How many Person-Days do you need when you can focus on this task?"

2. Convert from Estimator to Performer:
(see also CAP model, http://cap-model.com)

	Performer	Novice	Practitioner	Master	Expert	Guru
	Non-Linear Effort Reduction	0%	10%	25%	45%	80%
Estimator	Novice	1,00	0,90	0,75	0,55	0,20
	Practitioner	1,11	1,00	0,83	0,61	0,22
	Master	1,33	1,20	1,00	0,73	0,27
	Expert	1,82	1,64	1,36	1,00	0,36
	Guru	5,00	4,50	3,75	2,75	1,00

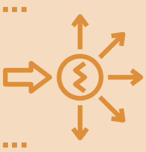


Sizes & Variability

Estimation Sizes and Estimation Variability:

T-Shirt-Size (Logically)	XXS	XS	S	M	L	XL	XXL	XXXL
Fibonacci-Size (PD or SP)	**0,50**	**1**	**2**	**3**	**5**	**8**	**13**	**21**
Size Variability (-)	0,25	0,25	0,50	0,50	1,00	1,50	2,50	4,00
Size Variability (+)	0,25	0,50	0,50	1,00	1,50	2,50	4,00	8,00

Notice: Estimations can be done in *Person-Days (PD)* or *Story-Points (SP)*. In both cases, keep in mind to use something like the *Fibonacci* numbers which increase in a non-linear fashion and express the increasing variability with the increasing total amount of estimated effort.



Risk Mitigation & Upscaling

3. Adjust for Reality:
Estimator Optimism: **+30%**
Performer Meetings: **+20%**

4. Adjust for Uncertainty:

Domain	Inception	Elaboration	Construction
unknown	30%	40%	20%
partially known	15%	20%	10%
fully known	0%	0%	0%

Technology	Inception	Elaboration	Construction
unknown	20%	60%	10%
partially known	10%	30%	5%
fully known	0%	0%	0%

Process	Inception	Elaboration	Construction
unknown	60%	40%	10%
partially known	30%	20%	5%
fully known	0%	0%	0%

People	Inception	Elaboration	Construction
unknown	60%	40%	0%
partially known	30%	20%	0%
fully known	0%	0%	0%

Intellectual Content: Version 1.0.1 (2022-10-10), Authored 2021-2022 by Dr. Ralf S. Engelschall based on concepts from Steve McDonnell
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

uncertainty & efforts

Effort estimations are usually based on a **Three-Point Estimation** where a weighted average of "best case", "most likely" and "worst case" is used. A good estimation variability in practice is about +/- 20%.

For **Expert Estimations**, a fixed scale of estimation sizes is usually used in practice, which is based on the **Fibonacci** sequence of numbers, to take into account the fact that higher estimated efforts also have higher estimation variability.

Additionally, one usually has to post-adjust the estimation of experts to further take into account the different skill and experience levels between the task estimator and the subsequent task performer, the usual human optimism of the estimator and the practical meeting and inevitable communication distractions of the performer.

In case of uncertainty because of entirely unknown or at least just partially known aspects Domain, Technology, Process and People, the total estimated efforts of the usual project phases have to be additionally upscaled.

Questions

? What variability does a **good** estimation have?

ENGINEERING
FUNDAMENTALS

Agentic Software Engineering

TECHNISCHE
UNIVERSITÄT
MÜNCHEN

SW **Software** 1

The collection of programs (instructions) and their associated data that make a computer perform tasks. In contrast to hardware, it is intangible and can be changed, copied, and distributed without altering the physical machine.

SA **Software Agent** 2

Software that acts autonomously on behalf of a user or another program to achieve delegated goals. Exhibits reactivity (responds to environment), proactivity (goal-directed initiative), and social ability (interacts with other agents/users). Perceives its environment via sensors/inputs (APIs, files, messages, events) and acts via actuators/outputs (commands, calls, messages).

EN **Engineering** 3

The systematic application of scientific and technological knowledge, principles, approaches, practices, methods, and experiences to design, build, operate, and maintain solutions, up to complex production-grade ones, through a traceable process.

SE **Software Engineering** 4

The application of **Engineering** to the development, operation, and maintenance of **Software**.

AI **Artificial Intelligence** 5

The field of Computer Science focusing on building systems that perform tasks normally requiring human intelligence: perception, reasoning, learning, planning, decision-making, and language understanding. It spans Symbolic AI (rules, logic, search) and Machine Learning (including Deep Learning and Generative AI such as LLMs and diffusion models), ranging from narrow, task-specific AI to Artificial General Intelligence (AGI).

AH **Agent Harness** 6

The **Software** that surrounds an AI model and turns it from a passive, predicting model into an acting **Software Agent**. It implements a control loop (e.g. ReAct or Plan-and-Execute) that sends context to the model, executes the requested actions, and feeds results back, and it supplies the tools (sensors and actuators), context management, and guardrails. The model decides what to do; the harness determines what it can do and how.

AG **AI Agent** 7

A **Software Agent** formed by an AI model (typically an Large Language Model, LLM) as its reasoning core, operated by an **Agent Harness**.

AA **Agentic AI** 8

The paradigm of building AI systems out of one or more **AI Agents** that perceive, plan, decide, act, and adapt to pursue high-level, long-horizon goals with a high degree of autonomy, typically coordinated through task decomposition, orchestration, and shared context. Unlike purely generative (prompt-response) AI, its systems take actions in the world (call APIs, run code, edit files, browse the web).

AS **Agentic Software Engineering** 9

Software Engineering following the **Agentic AI** paradigm, in which **AI Agents** perform multi-step tasks across the software development lifecycle, within a traceable process. Goes beyond inline code completion and chat assistants. Engineers shift toward goal specification, oversight, review, and governance, with human-in-the-loop gates.

Foundations: Software is the intangible, changeable set of programs and data that make a computer do tasks. Engineering is the systematic, traceable application of scientific knowledge to design, build, operate, and maintain solutions, so Software Engineering applies that discipline across a software system's lifecycle.

From AI to agents: Artificial Intelligence ranges from symbolic AI to machine learning and generative models like LLMs. A Software Agent acts autonomously toward delegated goals: it perceives its environment through inputs and acts through outputs, reactively, proactively, and socially. An AI Agent is such an agent with an AI model (usually an LLM) as its reasoning core. The Agent Harness around the model runs the control loop (e.g., ReAct), supplies tools, manages context, and enforces guardrails. The model decides what to do; the harness decides what it can do.

Agentic Software Engineering: Agentic AI builds systems of autonomous agents that plan, act, and adapt toward long-horizon goals, taking real actions rather than just generating responses. Applied to Software Engineering, agents carry out multi-step tasks across the development lifecycle within a traceable process. Human engineers shift toward specifying goals, overseeing and reviewing the work, and governing it through Human-in-the-Loop gates.

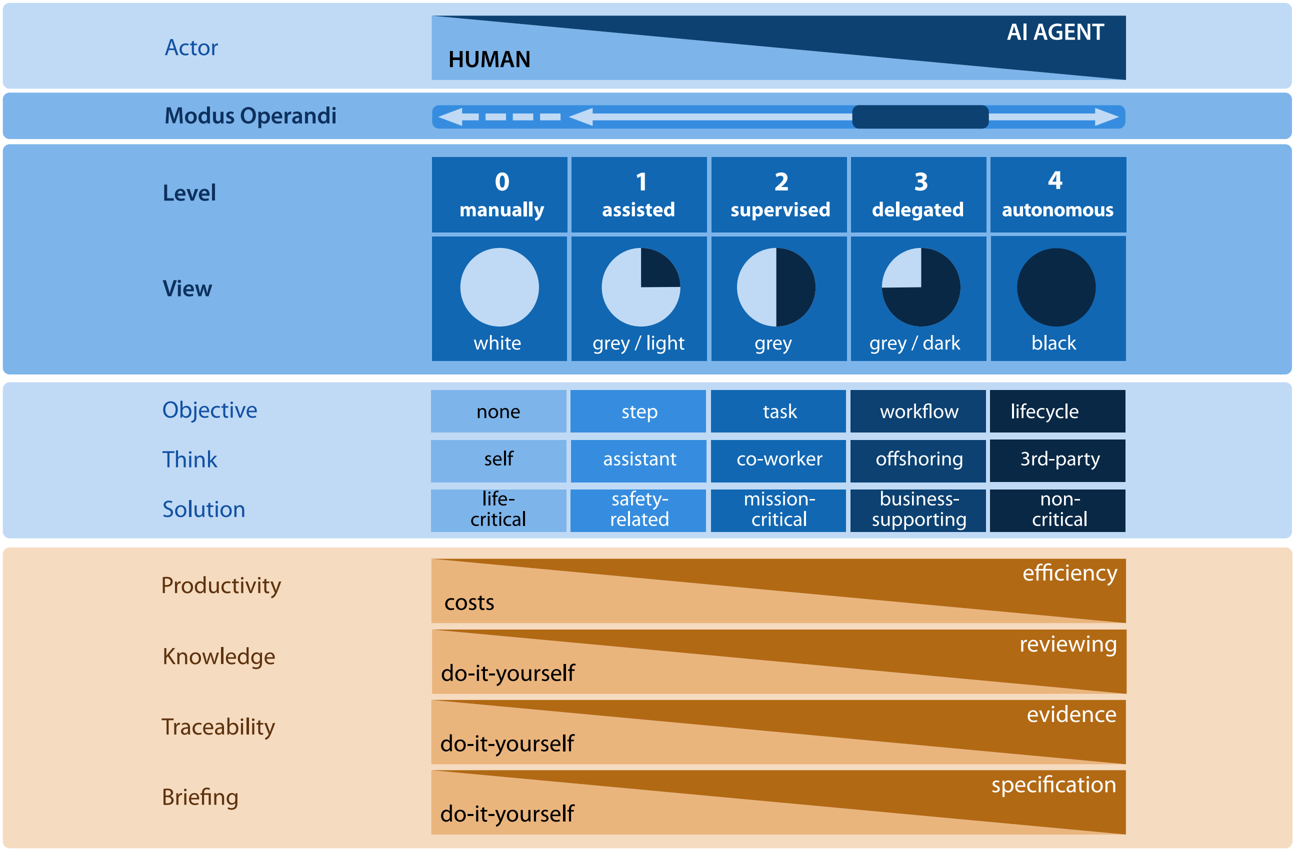
Questions

- ❓ How is the work divided between the AI model and the agent harness in an AI agent?

Intellectual Content: Version 1.0.0 (2026-10-28), Authored 2026 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2026-10-28), Copyright © 2026 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

concepts & relationships

Agentic Modus Operandi



Level	0 manually	1 assisted	2 supervised	3 delegated	4 autonomous
View	white	grey / light	grey	grey / dark	black
Objective	none	step	task	workflow	lifecycle
Think	self	assistant	co-worker	offshoring	3rd-party
Solution	life-critical	safety-related	mission-critical	business-supporting	non-critical

Productivity	costs	efficiency
Knowledge	do-it-yourself	reviewing
Traceability	do-it-yourself	evidence
Briefing	do-it-yourself	specification

Tasks are shared between human and AI agent across five autonomy levels: manually (0), assisted (1), supervised (2), delegated (3), and autonomous (4). Each level matches a familiar way of working: doing it yourself, using an assistant, working with a co-worker, offshoring, and finally handing it to a third party. Along the way, the agent's work result shifts from a white box (fully visible and comprehensible) to a black box (opaque).

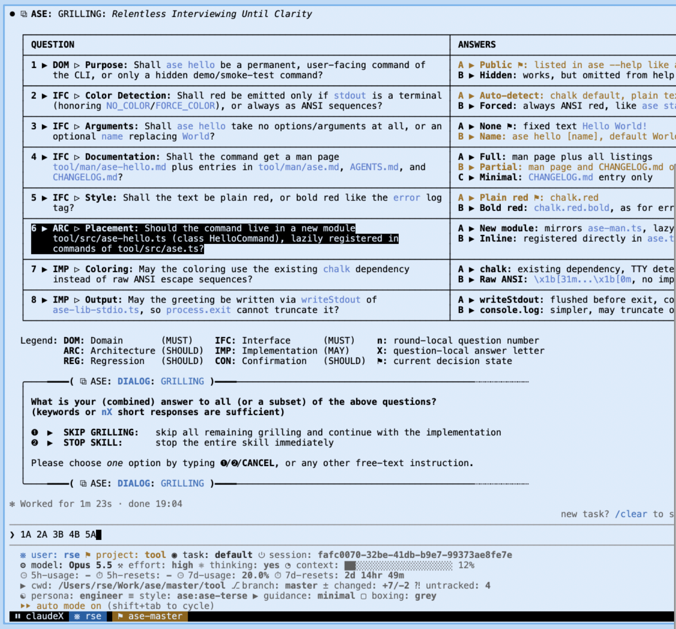
With each level, the agent takes on a larger objective, from none to a single step, a task, a workflow, and up to the entire lifecycle. Higher autonomy fits less critical solutions, ranging from life-critical and safety-related systems at the manual end to supporting and non-critical systems at the autonomous end.

Moving toward autonomy shifts productivity from human costs toward efficiency. The human role changes too: knowledge moves from doing it yourself to reviewing, traceability from doing it yourself to evidence, and briefing from doing it yourself to precise specification.

Questions

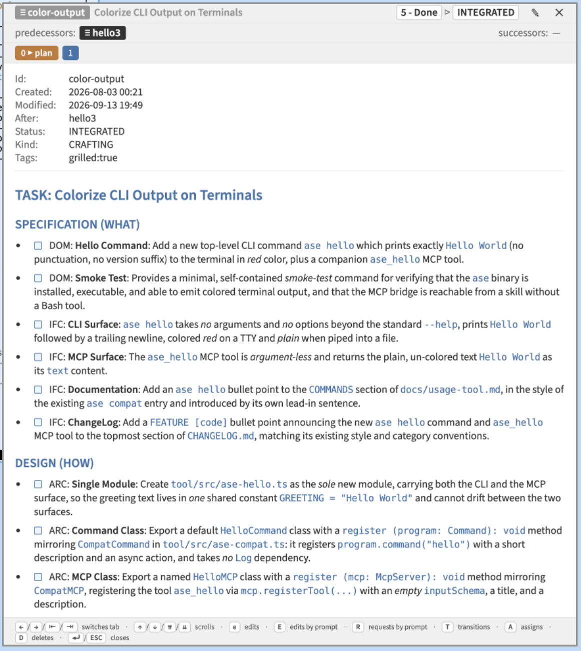
- ? At which autonomy level does an agent operate that independently carries out a complete workflow?

Agentic Briefing



```
● ⧉ ASE: GRILLING: Relentless Interviewing Until Clarity

  ┌──────────────────────────────────────────────────────────────┬──────────────────────────────────────────────────────
  │ QUESTION                                                     │ ANSWERS
  ├──────────────────────────────────────────────────────────────┼──────────────────────────────────────────────────────
  │ 1 ▶ DOM ▷ Purpose: Shall ase hello be a permanent, user-facing command of │ A ▶ Public ⚑: listed in ase --help like all others
  │     the CLI, or only a hidden demo/smoke-test command?      │ B ▶ Hidden: works, but omitted from help output
  ├──────────────────────────────────────────────────────────────┼──────────────────────────────────────────────────────
  │ 2 ▶ IFC ▷ Color Detection: Shall red be emitted only if stdout is a terminal │ A ▶ Auto-detect: chalk default, plain text on pipes
  │     (honoring NO_COLOR/FORCE_COLOR), or always as ANSI sequences? │ B ▶ Forced: always ANSI red, like ase statusline
  ├──────────────────────────────────────────────────────────────┼──────────────────────────────────────────────────────
  │ 3 ▶ IFC ▷ Arguments: Shall ase hello take no options/arguments at all, or an │ A ▶ None ⚑: fixed text Hello World!
  │     optional name replacing World?                           │ B ▶ Name: ase hello [name], default World
  ├──────────────────────────────────────────────────────────────┼──────────────────────────────────────────────────────
  │ 4 ▶ IFC ▷ Documentation: Shall the command get a man page   │ A ▶ Full: man page plus all listings
  │     tool/man/ase-hello.md plus entries in tool/man/ase.md, AGENTS.md, and │ B ▶ Partial: man page and CHANGELOG.md o
  │     CHANGELOG.md?                                            │ C ▶ Minimal: CHANGELOG.md entry only
  ├──────────────────────────────────────────────────────────────┼──────────────────────────────────────────────────────
  │ 5 ▶ IFC ▷ Style: Shall the text be plain red, or bold red like the error log │ A ▶ Plain red ⚑: chalk.red
  │     tag?                                                     │ B ▶ Bold red: chalk.red.bold, as for err
  ├──────────────────────────────────────────────────────────────┼──────────────────────────────────────────────────────
  │ 6 ▶ ARC ▷ Placement: Should the command live in a new module │ A ▶ New module: mirrors ase-man.ts, lazy
  │     tool/src/ase-hello.ts (class HelloCommand), lazily registered in │ B ▶ Inline: registered directly in ase.t
  │     commands of tool/src/ase.ts?                             │
  ├──────────────────────────────────────────────────────────────┼──────────────────────────────────────────────────────
  │ 7 ▶ IMP ▷ Coloring: May the coloring use the existing chalk dependency │ A ▶ chalk: existing dependency, TTY dete
  │     instead of raw ANSI escape sequences?                   │ B ▶ Raw ANSI: \x1b[31m...\x1b[0m, no imp
  ├──────────────────────────────────────────────────────────────┼──────────────────────────────────────────────────────
  │ 8 ▶ IMP ▷ Output: May the greeting be written via writeStdout of │ A ▶ writeStdout: flushed before exit, co
  │     ase-lib-stdio.ts, so process.exit cannot truncate it?   │ B ▶ console.log: simpler, may truncate o
  └──────────────────────────────────────────────────────────────┴──────────────────────────────────────────────────────

  Legend: DOM: Domain       (MUST)    IFC: Interface      (MUST)    n: round-local question number
          ARC: Architecture (SHOULD)  IMP: Implementation (MAY)     X: question-local answer letter
          REG: Regression   (SHOULD)  CON: Confirmation   (SHOULD)  ⚑: current decision state

  ┌────( ⧉ ASE: DIALOG: GRILLING )──────────────────────────────────────────────────

  What is your (combined) answer to all (or a subset) of the above questions?
  (keywords or nX short responses are sufficient)

  ❶ ▶  SKIP GRILLING:   skip all remaining grilling and continue with the implementation
  ❷ ▶  STOP SKILL:      stop the entire skill immediately

  Please choose one option by typing ❶/❷/CANCEL, or any other free-text instruction.

  └────( ⧉ ASE: DIALOG: GRILLING )──────────────────────────────────────────────────

✻ Worked for 1m 23s · done 19:04
                                                                  new task? /clear to s
─────────────────────────────────────────────────────────────────────────────────────────
❯ 1A 2A 3B 4B 5A█
─────────────────────────────────────────────────────────────────────────────────────────
  ⋇ user: rse ⚑ project: tool ◉ task: default ⏻ session: fafc0070-32be-41db-b9e7-99373ae8fe7e
  ⚙ model: Opus 5.5 ⚒ effort: high ❋ thinking: yes ◔ context: █░░░░░░░░░░░░░░░░░░░░ 12%
  ◷ 5h-usage: – ⏱ 5h-resets: – ◷ 7d-usage: 20.0% ⏱ 7d-resets: 2d 14hr 49m
  ▶ cwd: /Users/rse/Work/ase/master/tool ⎇ branch: master ± changed: +7/-2 ⁈ untracked: 4
  ◕ persona: engineer ≡ style: ase:ase-terse ▶ guidance: minimal ▢ boxing: grey
  ▸▸ auto mode on (shift+tab to cycle)
 ▮ claudeX  ⋇ rse  ⚑ ase-master
```

Brief Engineering: The systematic specification of a delegated assignment for an AI agent: goals, scope, constraints, acceptance criteria, and safety boundaries. It defines *what* must be achieved and how success is verified, so an agent can work autonomously across long-horizon tasks and deliver traceable evidence.



predecessors: ≡ hello3 successors: —

Id: color-output
Created: 2026-08-03 00:21
Modified: 2026-09-13 19:49
After: hello3
Status: INTEGRATED
Kind: CRAFTING
Tags: grilled:true

TASK: Colorize CLI Output on Terminals

SPECIFICATION (WHAT)

- ☐ DOM: **Hello Command**: Add a new top-level CLI command `ase hello` which prints exactly `Hello World` (no punctuation, no version suffix) to the terminal in *red* color, plus a companion `ase_hello` MCP tool.
- ☐ DOM: **Smoke Test**: Provides a minimal, self-contained *smoke-test* command for verifying that the `ase` binary is installed, executable, and able to emit colored terminal output, and that the MCP bridge is reachable from a skill without a Bash tool.
- ☐ IFC: **CLI Surface**: `ase hello` takes *no* arguments and *no* options beyond the standard `--help`, prints `Hello World` followed by a trailing newline, colored *red* on a TTY and *plain* when piped into a file.
- ☐ IFC: **MCP Surface**: The `ase_hello` MCP tool is *argument-less* and returns the plain, un-colored text `Hello World` as its `text` content.
- ☐ IFC: **Documentation**: Add an `ase hello` bullet point to the `COMMANDS` section of `docs/usage-tool.md`, in the style of the existing `ase compat` entry and introduced by its own lead-in sentence.
- ☐ IFC: **ChangeLog**: Add a `FEATURE [code]` bullet point announcing the new `ase hello` command and `ase_hello` MCP tool to the topmost section of `CHANGELOG.md`, matching its existing style and category conventions.

DESIGN (HOW)

- ☐ ARC: **Single Module**: Create `tool/src/ase-hello.ts` as the *sole* new module, carrying both the CLI and the MCP surface, so the greeting text lives in *one* shared constant `GREETING = "Hello World"` and cannot drift between the two surfaces.
- ☐ ARC: **Command Class**: Export a default `HelloCommand` class with a `register (program: Command): void` method mirroring `CompatCommand` in `tool/src/ase-compat.ts`: it registers `program.command("hello")` with a short description and an async action, and takes *no* `Log` dependency.
- ☐ ARC: **MCP Class**: Export a named `HelloMCP` class with a `register (mcp: McpServer): void` method mirroring `CompatMCP`, registering the tool `ase_hello` via `mcp.registerTool(...)` with an *empty* `inputSchema`, a title, and a description.

←/→/⇤/⇥ switches tab · ↑/↓/⇈/⇊ scrolls · e edits · E edits by prompt · R requests by prompt · T transitions · A assigns · D deletes · ↵/ESC closes

Prompt Engineering: The craft of formulating individual inputs (prompts) to an AI model so that a single request reliably produces the desired output. It focuses on phrasing, structure, examples, and instructions within one prompt–response exchange.

Context Engineering: The systematic design and management of everything an AI model sees while it works: instructions, retrieved knowledge, tool definitions and results, memory, and conversation history. It decides what goes into the limited context window at each step of an agent's control loop, usually handled by the Agent Harness.

Three levels of engineering: Prompt Engineering optimizes a single prompt–response exchange. Context Engineering manages everything the model sees at each step of the agent's control loop. Brief Engineering specifies a whole delegated assignment: what must be achieved and how success is verified.

Grilling for clarity: "Grilling" means the agent relentlessly interviews the human until the brief is clear. In practice it usually is instructed to ask structured multiple-choice questions, prioritized according to Domain, Interface, Architecture, and Implementation.

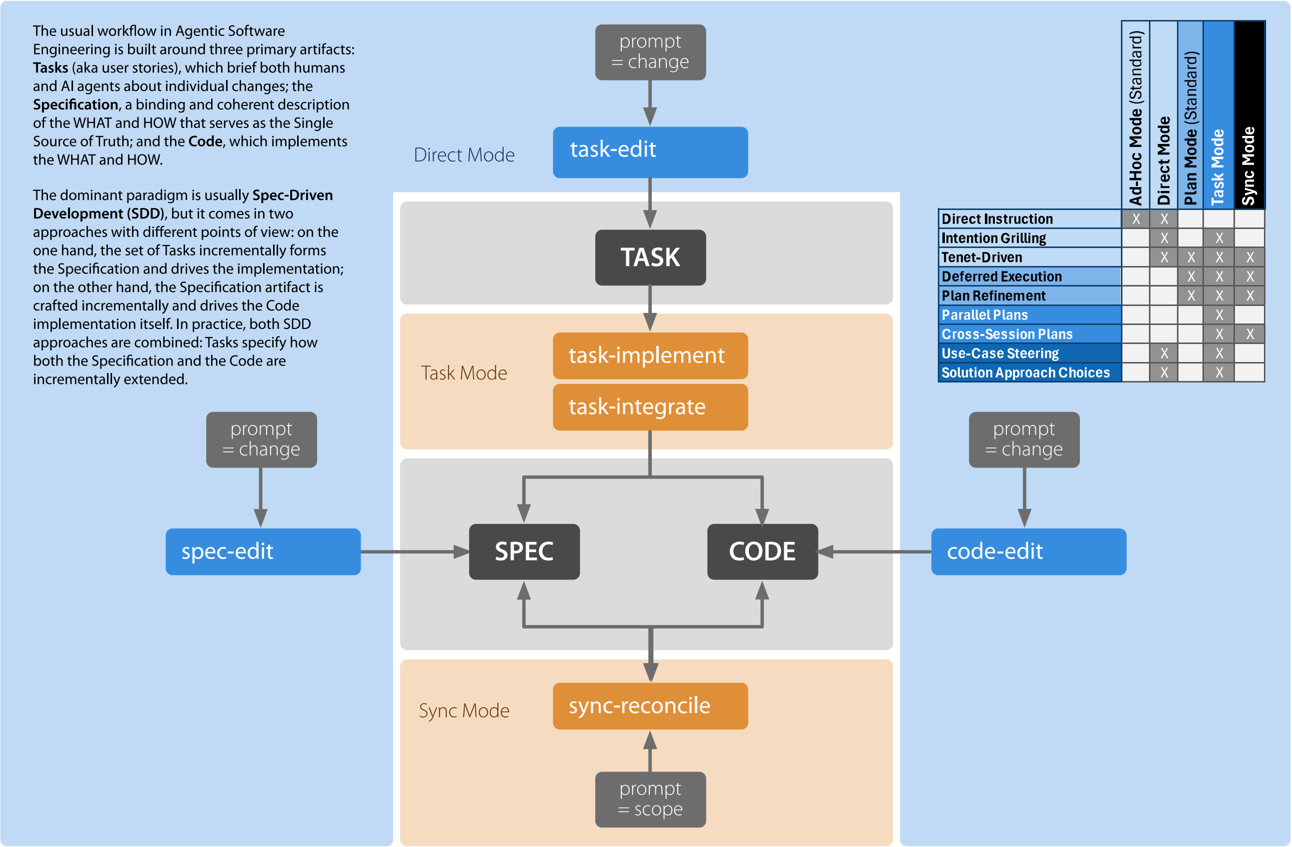
The answers usually become a structured task with checkable items, split into a Specification (WHAT), a Design (HOW), and a Verification (WHEN). This gives the agent a precise brief for autonomous work and gives the human a traceable record for review.

Questions

? What is meant by "grilling", and why is it especially important when the agent has high autonomy?

Agentic Workflow

concepts & relationships



The usual workflow in Agentic Software Engineering is built around three primary artifacts: **Tasks** (aka user stories), which brief both humans and AI agents about individual changes; the **Specification**, a binding and coherent description of the WHAT and HOW that serves as the Single Source of Truth; and the **Code**, which implements the WHAT and HOW.

The dominant paradigm is usually **Spec-Driven Development (SDD)**, but it comes in two approaches with different points of view: on the one hand, the set of Tasks incrementally forms the Specification and drives the implementation; on the other hand, the Specification artifact is crafted incrementally and drives the Code implementation itself. In practice, both SDD approaches are combined: Tasks specify how both the Specification and the Code are incrementally extended.

	Ad-Hoc Mode (Standard)	**Direct Mode**	**Plan Mode** (Standard)	**Task Mode**	**Sync Mode**
Direct Instruction	X	X			
Intention Grilling		X		X	
Tenet-Driven		X	X	X	X
Deferred Execution			X	X	X
Plan Refinement			X	X	X
Parallel Plans				X	
Cross-Session Plans				X	X
Use-Case Steering		X		X	
Solution Approach Choices		X		X	

The usual workflow in Agentic Software Engineering is built around three primary artifacts: **Tasks** (aka User Stories), which brief both humans and AI agents about individual changes; the **Specification**, a binding and coherent description of the WHAT and HOW that serves as the Single Source of Truth; and the **Code**, which implements the WHAT and HOW.

The dominant paradigm is usually **Spec-Driven Development (SDD)**, but it comes in two approaches with different points of view: on the one hand, the set of Tasks incrementally forms the Specification and drives the implementation; on the other hand, the Specification artifact is crafted incrementally and drives the Code implementation itself. In practice, both SDD approaches are combined: Tasks specify how both the Specification and the Code are incrementally extended.

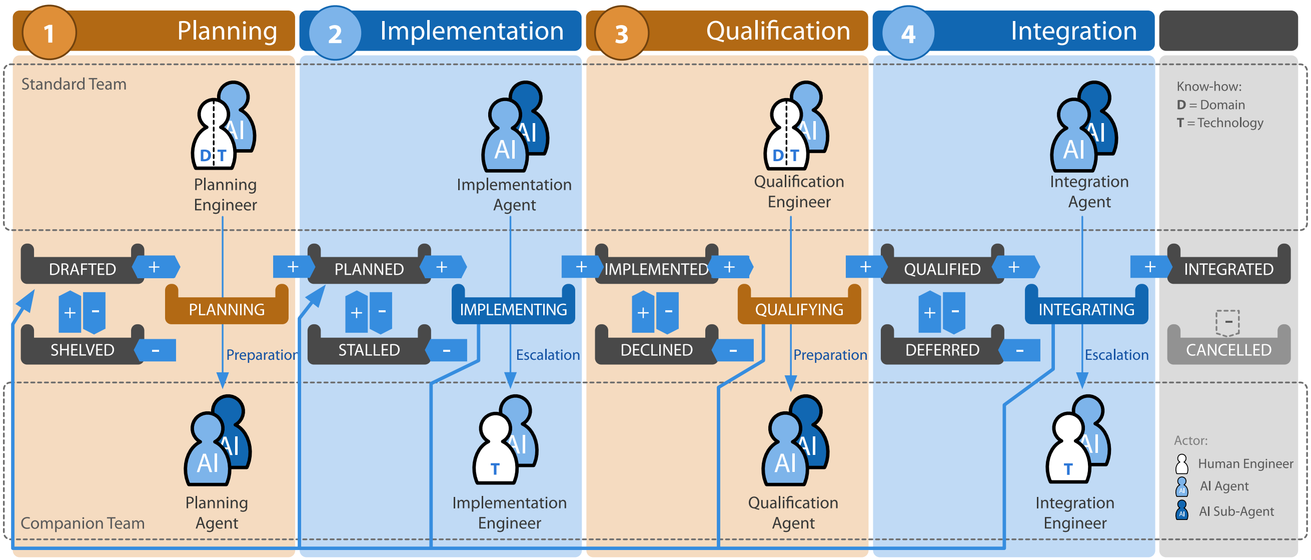
Questions

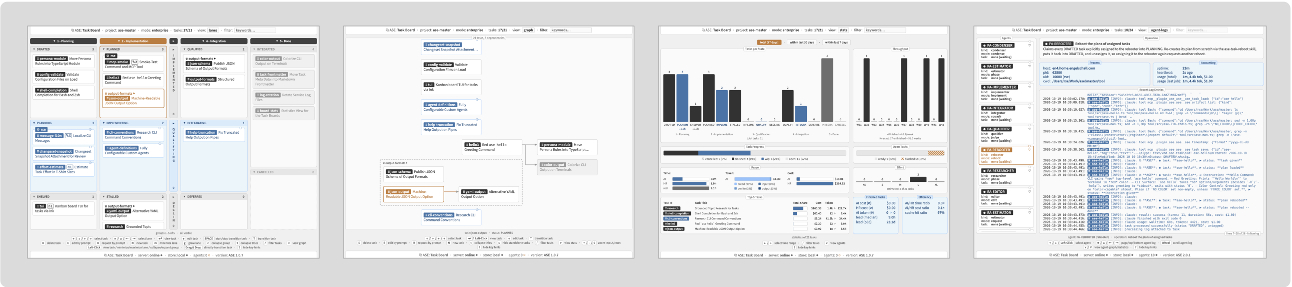
- What are the three primary artifacts in Agentic Software Engineering, and how are the two SDD approaches combined in practice?

ENGINEERING FUNDAMENTALS

Agentic Development Pipeline

TECHNISCHE UNIVERSITÄT MÜNCHEN





A Task passes through four phases: Planning, Implementation, Qualification, and Integration. Its main states are DRAFTED, PLANNED, IMPLEMENTED, QUALIFIED, and INTEGRATED, connected by the active states PLANNING, IMPLEMENTING, QUALIFYING, and INTEGRATING. Each phase also has a parking state (SHELVED, STALLED, DECLINED, DEFERRED), tasks can be CANCELLED, and failed tasks loop back to earlier phases.

The Standard Team consists of humans and AI agents who alternate across the phases. Human engineers with domain and technology know-how handle Planning and Qualification, where judgment about the WHAT and its acceptance matters. AI agents handle Implementation and Integration, where execution of the HOW dominates.

The Companion Team complements this setup: each primary actor is backed by a counterpart. AI agents do the preparation for the human Planning and Qualification Engineers, while human engineers with technology know-how take over escalations from the Implementation and Integration Agents.

Questions

- ? Why do humans handle Planning and Qualification in the Standard Team, while AI agents handle Implementation and Integration?